FINAL PROJECT SPECIFICATION DOCUMENT

COMP6703001 – Web Application Development and Security

Program	**:**	Computer Science
Institution	**:**	BINUS University International
Course Code	**:**	COMP6703001
Course Name	**:**	Web Application Development and Security
Semester	**:**	Even 2025-2026
Instructors	**:**	(Main) - Ida Bagus Kerthyayana Manuaba (Lab) - Juwono

1. Final Project Overview

In this course, you will design, develop, test, secure, and deploy a **full-stack web application** that includes **AI-powered functionality**.

The final project is **one continuous project** developed progressively throughout the semester.
Each week, you will implement features that directly correspond to the topics taught in class.

This project is designed to ensure that you can:

- Build real-world web applications
- Apply backend, frontend, database, API, and security concepts
- Integrate AI responsibly
- Test your system properly, including AI behavior
- Deploy a production-ready application

You are expected to apply **web development, API design, security, testing, AI integration, and production deployment** in one coherent system.

This is not a prototype or mock-up project. Your application must be **production-ready**.

2. Project Format

2.1 Group Requirement (IMPORTANT)

- **Group size:**
  - Maximum **3 students**
  - **Within the same class only** (cross-class groups are NOT allowed)
- All group members must:

- Contribute code
- Understand the full system
- Be able to explain any part during presentation

2.2 Mandatory Technology Stack (NO EXCEPTIONS)

You **must** use the following technologies:

A. Frontend

- **Next.js (React framework)**
  - App Router or Pages Router allowed
  - Client–server rendering concepts must be demonstrated

B. Backend

- **Node.js**
- Backend logic may be implemented:
  - inside Next.js API routes **or**
  - as a separate Node.js service

C. API (MANDATORY)

- RESTful API is **required**
- API must:
  - follow HTTP standards (GET, POST, PUT, DELETE)
  - return JSON responses
  - be documented

❗ Applications without proper API design will fail the backend requirement.

D. Database Requirement

You must use **one** of the following:

- **PostgreSQL with PRISMA**
- **Firebase Services (Firestore / Auth) - optional**

Requirements:

- Proper schema or data structure
- Secure connection
- Database must NOT be accessed directly from frontend (except for auth)

E. Deployment & Production Requirement (MANDATORY)

Your application **must be deployed** and publicly accessible.

E.1 Containerization

- **Docker is mandatory**
- You must provide:
    - Dockerfile
    - docker-compose.yml (if applicable)

E.2 Production Setup

You are expected to **learn and implement**:

- Environment variables
- Production vs development configuration
- Secure secrets handling
- HTTPS (where applicable)
- Setting Domain Configuration using Cloudflare (Provide by Lab Instructur)

Local-only projects will receive **major penalties**.

F. Version Control Requirement

- **GitHub is mandatory**
- Repository must include:
    - Meaningful commit history
    - Branching (at least main + feature branches)
    - README (Final Project Report + setup & deployment instructions)

3. Approved Project Options

You must choose **ONE** of the following project domains:

1. Language Learning Web Application
2. Career Counseling & Guidance Application
3. Educational Games & Quiz Platform
4. Inclusive Education Support Application
5. Homework Question & Answer Assistant
6. Interactive Essay Grading Application
7. Virtual Science Lab Simulator
8. Student Community & Collaboration Platform
9. Math Problem Solver Application
10. Study Planner & Productivity Tracker

Each project **must** include:

- Full-stack implementation

- API usage
- Database integration
- AI functionality
- Security controls
- Testing documentation
- Docker deployment

Detail Features available in **Appendix A**

4. General System Requirements (All Projects)

4.1 Frontend Requirements

- Responsive UI (desktop & mobile)
- Built using modern frontend frameworks (React, Vue, etc.)
- Form validation and error handling
- Proper API integration

4.2 Backend Requirements

- RESTful API architecture
- Proper routing and controllers
- Business logic separated from routes
- Environment-based configuration

4.3 Database Requirements

- Well-designed schema (ERD required)
- Persistent storage for users and project data
- Secure access from backend only

5. AI Functionality Requirements (Mandatory)

Each project must implement **at least TWO meaningful AI features** relevant to the chosen domain.

Examples:

- NLP-based analysis or generation
- Recommendation systems
- Adaptive logic

- OCR / speech recognition
- Intelligent moderation or classification

AI features must be:

- Core to the application
- Testable
- Securely integrated

❗ AI must be **part of core functionality**, not decoration.

6. Security Requirements (Mandatory)

Your application must implement:

- Secure authentication (JWT / session)
- Authorization (role-based access)
- Input validation and output sanitization
- Protection against:
    - SQL / NoSQL Injection
    - Cross-Site Scripting (XSS)
    - Cross-Site Request Forgery (CSRF)
- Secure API key handling
- Proper session and cookie configuration

Security must be **demonstrated**, not just claimed, It **must be implemented progressively**, not only at the end.

7. Testing Requirements (Very Important)

Testing is a **core learning outcome** of this course.

7.1 System Testing (Required)

Each group must implement:

- Frontend testing
    - Form validation testing
    - UI behavior testing
    - Error handling tests

- Backend & API testing
    - API endpoint testing
    - Input and error case testing
- Integration testing (API ↔ database)
- Security Testing
    - XSS test attempts
    - Injection test attempts
    - Authentication and authorization testing

7.2 AI Functionality Testing (Mandatory)

AI features must be tested **as software components**. Each AI feature must include **documented test cases**, covering:

1. **Input variation testing**
    - valid input
    - invalid input
    - edge cases
2. **Expected output definition**
3. **Consistency testing**
4. **Failure handling**
    - timeout
    - AI unavailable
    - malformed response
5. **Abuse or misuse testing**
    - prompt injection
    - nonsensical input

📌 Simply saying "the AI works" is **not acceptable**.

8. Weekly Project Implementation Mapping

Your project must demonstrate the following progress:

Week	Required Progress	Submission Deadline
1	Architecture, tech stack, project selection	Submit 1 day before week 2 (via bimay lab session)
2	USR (User Persona,User Journey) and SRS (Architecture Diagram, UCD, Activity Diagram, Class Diagram)	Submit 1 day before week 3 (via bimay lab session)
3	Mockup design + Routing Design	Submit 1 day before week 4 (via bimay lab session)

4	Front End Testing	Submit 1 day before week 5 (via bimay lab session)
5	API Documentation (e.g. Swagger)	Submit 1 day before week 6 (via bimay lab session)
6	API Testing Result (e.g Manual Postman)	Submit 1 day before week 7 (via bimay lab session)
7	DB Schema and ORM	Submit 1 day before week 8 (via bimay lab session)
8	Auth and Authorization	Submit 1 day before week 9 (via bimay lab session)
9	Web Security (Input Vallidation, Middleware, Error Handling) + Security Testing	Submit 1 day before week 10 (via bimay lab session)
10	Dockerfile + Docker Compose + Github Action + Deployment Log Testing	Submit 1 day before week 11 (via bimay lab session)
11	AI Desain + Testing	Submit 1 day before week 12 (via bimay lab session)
12	Submit Final Project	Submit 2 day before week 13 (via bimay lab session)
13	Final demo & presentation	TBA - onsite

9. Final Deliverables

You must submit:

1. Git repository link
2. Live deployed application URL
3. Final report link (link to Readme file in your repository) containing:
    - Group Detail
    - Project Title and Description
    - Architecture Design (USR + SRS)
    - Features
    - Security implementation
    - Testing strategy (including AI)

    Note: Detail Template can be seen on Appendix B

4. API documentation link
5. Final project presentation video link (6-8 minutes) – youtube public access (Narrative explanation about the app and feature with your face and voice (+ caption) – No Code Explanation)

10. Assessment & Grading Rubrics

Total Final Score Weight: 100%

- **Group Score:** 60%
- **Individual Score:** 40%

⚠ Important
A strong group system **does not guarantee** a high individual score.
Each student must demonstrate **technical contribution and understanding**.

10.1 Group Assessment (60%)

Group assessment evaluates the **quality of the system as a whole**.

Group Rubric Summary

Category	Weight
System Architecture & Design	10%
Core Functionality & API	15%
Security Implementation	10%
AI Features & AI Testing	10%
Deployment & DevOps	10%
Documentation & Presentation	5%
Total	**60%**

A. System Architecture & Design (10%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
Architecture	Clear full-stack architecture, clean separation of concerns	Mostly clear with minor issues	Basic architecture, limited separation	Unclear or ad-hoc design
Design Pattern	Correct use of MVC / layered pattern	Partial pattern usage	Minimal structure	No clear structure

B. Core Functionality & API (15%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)

API Design	RESTful, consistent, documented	Mostly RESTful	Basic CRUD only	No proper API
Feature Completeness	All required features implemented	Minor missing features	Several incomplete features	Major functionality missing
Integration	Smooth frontend-API-DB integration	Minor integration bugs	Frequent issues	Not integrated

C. Security Implementation (10%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
Authentication	Secure, role-based, well implemented	Secure but limited roles	Basic login only	Insecure or missing
OWASP Mitigation	Multiple OWASP risks mitigated & explained	Some mitigation	Minimal mitigation	No security consideration
Secure API	Token handling, validation, sanitization	Partial	Weak validation	Vulnerable endpoints

D. AI Features & AI Testing (10%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
AI Relevance	AI is core and meaningful	AI useful but limited	AI superficial	AI irrelevant
AI Testing	Structured, documented, edge cases tested	Basic AI tests	Minimal AI testing	No AI testing
Failure Handling	Clear fallback & mitigation	Partial handling	Poor handling	No handling

⚠ **Important Rule:**

- Projects with AI features **but no structured AI testing** will be capped at **B**.
- No documented AI testing → Max B for this section

E. Deployment & DevOps (10%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
Docker	Clean Dockerfile(s), reproducible	Works but unoptimized	Bare minimum	No Docker
Production Setup	Env vars, prod config, secure	Partial setup	Weak setup	Local only
Live Deployment	Stable, accessible, HTTPS	Accessible with issues	Unstable	Not deployed

F. Documentation & Presentation (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
README	Clear setup, usage, deployment	Mostly clear	Minimal	Missing
Presentation	Clear demo, technical depth	Adequate	Surface-level	Cannot explain system

10.2 Individual Assessment (40%)

Individual assessment measures **personal contribution, understanding, and responsibility**.

Individual Rubric Summary

Category	Weight
Code Contribution (GitHub)	15%
Technical Understanding	10%
Individual Testing Contribution	10%
Professionalism & Responsibility	5%
Total	**40%**

A. Code Contribution (GitHub) – 15%

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)

Commit Quality	Meaningful, consistent commits	Regular commits	Few commits	No clear contribution
Ownership	Clear feature ownership	Partial ownership	Small contributions	Minimal contribution
Code Quality	Clean, readable, modular	Mostly clean	Messy but functional	Unusable

B. Technical Understanding – 10%

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
Explanation	Clearly explains own code	Explains most parts	Partial understanding	Cannot explain
Security Knowledge	Explains security decisions	Basic understanding	Vague answers	No understanding
AI Knowledge	Explains AI logic & limits	Partial explanation	Surface explanation	No understanding

C. Individual Testing Contribution – 10%

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
Test Design	Well-designed test cases	Adequate tests	Minimal tests	No tests
AI Testing	AI test cases written & explained	Basic AI tests	Weak AI tests	No AI testing
Bug Handling	Finds & fixes issues	Finds issues	Limited involvement	No involvement

D. Professionalism & Responsibility – 5%

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
Teamwork	Highly collaborative	Cooperative	Passive	Disruptive
Timeliness	Always on time	Mostly on time	Often late	Missed deadlines
Ethics	Honest, accountable	Minor issues	Questionable	Academic misconduct

10.3 Final Score Calculation

Final Project Score = (Group Score × 60%) + (Individual Score × 40%)

10.4 Important Enforcement Rules

Include this verbatim if you want **zero disputes** later:

- A student with **minimal GitHub contribution** cannot receive an A, regardless of group performance
- Missing **documented testing** will significantly reduce both group and individual scores
- AI features without testing will be capped at **B (max)**
- Non-deployed projects cannot receive higher than **C**

10.5 BONUS Score

- Score A automatically be invited to submit for Copyright Certification Supported by BINUS + Extra Score 20 points for Final Exam
- Group that could transform the final report into publication format ready (template provided and guide by instructors) with minimum score B+ could get a waiver for the Final Exam with Minimal **80 points** (submission deadline two days before Final Exam Schedule and get acknowledgement by the Instructor)

11. Academic Integrity & Scope Control

- Do not copy existing platforms
- AI tools may be used, but **you must understand and explain your code**
- Over-scoped projects will be penalized
- Simplicity + correctness better than feature overload

12. Policy on AI Usage (MANDATORY TO READ)

AI tools are **allowed and encouraged** in this course, **but only when used responsibly**.

This policy exists to ensure that:

- Students **learn**, not outsource thinking
- AI is treated as a **software component**, not a shortcut
- Academic integrity is maintained

12.1 Allowed Use of AI (PERMITTED)

Students **may use AI tools** for:

A. Development Support

- Code suggestions or refactoring
- Debugging assistance
- Explaining error messages
- Boilerplate generation (e.g., basic API setup)

B. AI as a System Feature

- NLP, OCR, recommendation systems, classification, etc.
- AI must be **integrated into the application**
- AI behavior must be **tested and documented**

C. Learning & Understanding

- Understanding concepts (e.g., authentication, Docker, security)
- Exploring alternative implementations
- Studying examples

12.2 Prohibited Use of AI (NOT PERMITTED)

The following are **strictly prohibited**:

❌ Submitting AI-generated code **without understanding it**
❌ Copy-pasting AI outputs that students cannot explain
❌ Using AI to generate:
- Entire projects
- Complete reports
- Final presentations without personal input

❌ Asking AI to:
- Bypass security
- Write malicious code
- Evade testing or plagiarism detection

❌ Claiming AI-generated output as personal work without disclosure

12.3 Disclosure Requirement (MANDATORY)

All groups **must disclose AI usage** in the final report.

The disclosure must include:

- AI tools used (e.g., ChatGPT, GitHub Copilot, OpenAI API)
- Purpose of usage (coding, testing, AI feature, documentation)
- Which parts of the system were assisted by AI

📌 Example disclosure (acceptable):

"ChatGPT was used to assist in generating initial API route structure and to brainstorm AI testing scenarios. All generated code was reviewed, modified, and understood by the team."

12.4 Understanding & Explanation Rule

During presentation or evaluation, any student **may be asked** to explain:

- Code logic
- Security decisions
- AI behavior
- Testing results

❗ Inability to explain your own code will result in **individual score penalties**, regardless of group performance.

12.5 AI Feature Accountability

If your project includes AI features, you **must**:

- Define expected AI behavior
- Test AI output systematically
- Handle AI failure cases
- Acknowledge AI limitations and risks

AI is treated as:
A probabilistic system that must be tested, monitored, and controlled

12.6 Academic Integrity & Consequences

Violations of this AI policy may result in:

- Score reduction
- Zero for the affected component
- Further academic integrity action according to BINUS regulations

12.7 Reminder to Students on AI related utilization

AI is a **tool**, not a replacement for learning.

If you can:

- explain your system,
- defend your security choices,
- justify your AI behavior,
- and show your testing,

then you are using AI **correctly**.

13. Final Note to Students

This project is designed to simulate **real-world web development**.
You are expected to think like:

- a developer,
- a security engineer,
- and a tester.

Build responsibly.

Appendix A – Project Options and Required Features

Students (Each Group) must select **ONE** project option and implement **ALL Core Features**, **AT LEAST TWO AI Features**, and **ALL Security Features** listed for that option.

---

1. Language Learning Web Application

Core Functional Features

- User registration and login
- Language selection (minimum 2 languages)
- Lesson modules (vocabulary, grammar, listening, speaking)
- Progress tracking dashboard
- Daily or weekly learning goals
- Quiz and assessment system
- User profile and learning history

AI-Driven Features

- Pronunciation analysis using speech-to-text
- Adaptive lesson difficulty based on learner performance
- AI-generated practice sentences or dialogues
- Automated feedback on pronunciation or grammar

Security-Critical Features

- Secure authentication (JWT or OAuth)
- Role separation (learner vs admin/content manager)
- Protection against XSS in user-generated inputs
- Rate-limiting on speech or text submissions
- Secure storage of learning data

---

2. Career Counseling & Guidance Application

Core Functional Features

- User profile with education, interests, and skills
- Career assessment questionnaire
- Career recommendation results page
- Skill gap visualization

- Saved career paths and learning plans
- Admin panel for updating career datasets

AI-Driven Features

- AI-based career matching engine
- Skill gap analysis and recommendation
- Resume or profile keyword analysis
- Predictive job trend insights (simplified model acceptable)

Security-Critical Features

- Secure handling of personal and career data
- Access control for admin-only data
- Input validation for assessments
- Secure API consumption for external job data
- Logging access to sensitive profile data

---

3. Educational Games & Quiz Platform

Core Functional Features

- Quiz/game creation interface
- Question bank with multiple categories
- Timed quizzes and scoring system
- Leaderboard and user ranking
- Student performance analytics
- Teacher/admin dashboard

AI-Driven Features

- Adaptive quiz difficulty
- AI-generated quiz questions
- Performance-based learning recommendations
- Automatic feedback explanation generation

Security-Critical Features

- Role-based access (student vs teacher)
- Anti-cheating measures (timing, session checks)
- Secure leaderboard updates
- Input sanitization for question creation

- Secure storage of scores and attempts

---

4. Inclusive Education Support Application

Core Functional Features

- Accessible UI (WCAG-aware design)
- Customizable font size, contrast, and layout
- Content upload and management
- Learning module navigation
- User accessibility preferences

AI-Driven Features

- Text-to-speech and speech-to-text
- Image or object description generation
- Automatic captioning for videos
- AI-assisted reading simplification

Security-Critical Features

- Secure storage of accessibility preferences
- Protection of uploaded content
- Permission-based content access
- Safe handling of media uploads
- Audit logs for content changes

---

5. Homework Question & Answer Assistant

Core Functional Features

- Question submission (text and image)
- Subject classification (math, science, etc.)
- Answer display with step-by-step explanation
- Question history and bookmarking
- User feedback on answer quality

AI-Driven Features

- NLP-based question understanding

- OCR for image-based questions
- AI-generated explanations
- Follow-up clarification chatbot

Security-Critical Features

- Rate limiting to prevent abuse
- Secure handling of uploaded images
- Input sanitization for text queries
- Protection against prompt injection
- Logging and monitoring of AI requests

---

6. Interactive Essay Grading Application

Core Functional Features

- Essay submission system
- Rubric configuration (criteria, weightings)
- Automated grading output
- Inline feedback and annotations
- Revision history tracking

AI-Driven Features

- AI-based scoring aligned to rubric
- Grammar and style analysis
- Argument structure evaluation
- Plagiarism similarity detection (simplified acceptable)

Security-Critical Features

- Secure document upload handling
- Role separation (student vs instructor)
- Access control to grading results
- Secure storage of submitted essays
- Protection against data leakage

---

7. Virtual Science Lab Simulator

Core Functional Features

- Experiment selection interface
- Interactive simulation controls
- Data capture and visualization
- Experiment result reports
- Teacher-defined experiment templates

AI-Driven Features

- AI-guided experiment hints
- Error detection in experiment setup
- Predictive outcome simulation
- Automated lab report feedback

Security-Critical Features

- Secure experiment data storage
- User isolation (no cross-data leaks)
- Input validation for simulation parameters
- Session integrity protection
- Secure file export (reports)

---

8. Student Community & Collaboration Platform

Core Functional Features

- User profiles and connections
- Discussion forums and threads
- Real-time or asynchronous chat
- File sharing and collaboration spaces
- Event or study session scheduling

AI-Driven Features

- AI content moderation
- Toxicity or spam detection
- Smart thread/topic recommendations
- Automated summarization of discussions

Security-Critical Features

- Content moderation enforcement
- File upload validation and scanning

- Role-based moderation controls
- Protection against XSS in messages
- Logging of moderation actions

---

9. Math Problem Solver Application

Core Functional Features

- Math problem input (text and image)
- Solution rendering interface
- Step-by-step explanation view
- Topic categorization (algebra, calculus, etc.)
- Practice problem suggestions

AI-Driven Features

- OCR for handwritten math
- Symbolic problem solving
- Hint generation instead of full answers
- Concept-based remediation suggestions

Security-Critical Features

- Secure image upload handling
- Rate limiting on problem submissions
- Input sanitization
- Prevention of model abuse
- Secure user activity tracking

---

10. Study Planner & Productivity Tracker

Core Functional Features

- Task and deadline management
- Calendar integration
- Study session timers
- Progress analytics dashboard
- Notifications and reminders

AI-Driven Features

- Smart task prioritization
- Study schedule optimization
- Productivity pattern detection
- Burnout or overload alerts

Security-Critical Features

- Secure calendar data handling
- Access control for personal schedules
- Protection of notification endpoints
- Encrypted storage of productivity data
- Session management and timeout controls

Instructor Note (Optional, Recommended to Include)

Important:
Projects that only implement basic CRUD without meaningful AI logic and security controls will receive **significant grade penalties**.

Appendix B. readme.md Template

Final Project – Web Application Development and Security

Course Code: COMP6703001
Course Name: Web Application Development and Security
Institution: BINUS University International

---

1. Project Information

Project Title:

[Your Project Name]

Project Domain (choose one):

Language Learning / Career Counseling / Quiz Platform / etc.

Class:

[Class Code] – L4AC/L4BC/L4CC

Group Members (Max 3 – same class only):

Name	Student ID	Role	GitHub Username

---

2. Instructor & Repository Access

This repository **must be shared** with:

- **Instructor:** Ida Bagus Kerthyayana Manuaba
  - Email: imanuaba@binus.edu
  - GitHub: bagzcode
- **Instructor Assistant:** Juwono
  - Email: juwono@binus.edu

- GitHub: *Juwono136*

---

3. Project Overview

3.1 Problem Statement

Explain:

- What problem does this application solve?
- Who are the target users?

3.2 Solution Overview

Briefly describe:

- Main features
- Why this solution is appropriate
- Where AI is used

---

4. Technology Stack (MANDATORY)

Layer	Technology
Frontend	Next.js
Backend	Node.js or Next.js
API	REST API
Database	PostgreSQL / Firebase (for auth only)
Containerization	Docker
Deployment	[Hosting Platform]
Version Control	GitHub

---

5. System Architecture

5.1 Architecture Diagram

Insert architecture diagram image or ASCII diagram here.

5.2 Architecture Explanation

Explain:

- Frontend ↔ API ↔ Database interaction
- Separation of concerns
- Where security is enforced

---

6. API Design (MANDATORY)

6.1 API Endpoints

Method	Endpoint	Description	Auth Required
GET			Yes / No
POST			Yes / No
PUT			Yes / No
DELETE			Yes / No

6.2 API Documentation

- Swagger / Postman link (if available)
- Example request & response (JSON)

---

7. Database Design

7.1 Database Choice

Explain why you chose:

- PostgreSQL / MongoDB / Firebase

7.2 Schema / Data Structure

Insert ERD or data structure diagram.

---

8. AI Features (MANDATORY)

8.1 AI Feature List

Describe **at least TWO AI features**.

AI Feature	Purpose	AI Type
		NLP / OCR / Recommendation
		NLP / OCR / Recommendation

8.2 AI Integration Flow

Explain:

- Input → AI processing → Output
- How AI results are used in the system

---

9. Security Implementation (MANDATORY)

Describe how your project handles:

- Authentication (JWT / session)
- Authorization (roles)
- Input validation
- Protection against:
  - SQL / NoSQL Injection
  - XSS
  - CSRF
- Secure API key handling

---

10. Testing Documentation (VERY IMPORTANT)

❗ **All testing must be documented.**

10.1 Frontend Testing

Test Case	Scenario	Expected Result	Status
FE-01			Pass / Fail
FE-02			Pass / Fail

10.2 Backend & API Testing

Test Case	Endpoint	Input	Expected Output	Status
API-01				Pass / Fail

10.3 Security Testing

Test Case	Attack Type	Expected Behavior	Result
SEC-01	XSS	Input sanitized	Pass
SEC-02	Injection	Query blocked	Pass

10.4 AI Functionality Testing (MANDATORY)

For **each AI feature**, complete the table below.

AI Feature: [Name]

Test Case	Input	Expected Output	Actual Result	Status
AI-01	Valid input	Correct response		Pass
AI-02	Invalid input	Error / fallback		Pass
AI-03	Prompt injection	Sanitized		Pass

Failure Handling:

- What happens if AI is unavailable?
- How is timeout handled?

---

11. Deployment & Production Setup

11.1 Docker Setup

- Dockerfile included ✔
- docker-compose.yml included ✔

11.2 Production Environment

Explain:

- Environment variables
- Secrets handling
- HTTPS configuration

11.3 Live Application URL

https://your-production-url.com

---

12. GitHub Contribution Summary (INDIVIDUAL)

Each student must list **their own contribution**.

Student Name: *[Name]*

- Features implemented:
- API endpoints handled:
- Tests written:
- Security work:
- AI-related work:

⚠ Contributions must match GitHub commit history.

---

13. AI Usage Disclosure (MANDATORY)

List:

- AI tools used (e.g., ChatGPT, OpenAI API)
- Purpose of usage
- Which parts were assisted

Example:

“ChatGPT was used to assist with API structure and AI testing scenario generation. All code was reviewed and modified by the team.”

---

14. Known Limitations & Future Improvements

- Current limitations
- Possible future enhancements
- AI limitations and risks

---

15. Final Declaration

We declare that:

- This project is our own work
- AI usage is disclosed honestly
- All group members understand the system

Signed by Group Members:

- _____
- _____
- _____

16. SETUP

17. DEPLOYMENT INSTRUCTIONS

Notes

- Missing sections in this README will directly impact grading.
- Incomplete testing or missing AI testing will cap the final grade.